

Industrial Equipment Monitoring Platform

Data Science & Scoring Layer Design

Technical Assessment

Aurik Technologies

Prepared By

Anushree Kalbandhe

August 2026

Problem Statement & Objectives

Problem Statement

Multiple industrial equipment vendors generate machine-level data in different schemas, timestamp formats, machine identifiers, and measurement units.

The objective is to build a unified machine-level data platform that validates, normalizes, scores, and exposes trusted equipment insights through backend APIs.

Key Challenges

- Different vendor schemas
- Different machine identifiers
- Different timestamp formats
- Different measurement units
- Missing and inconsistent values
- Conflicting sensor readings across vendors

Objectives

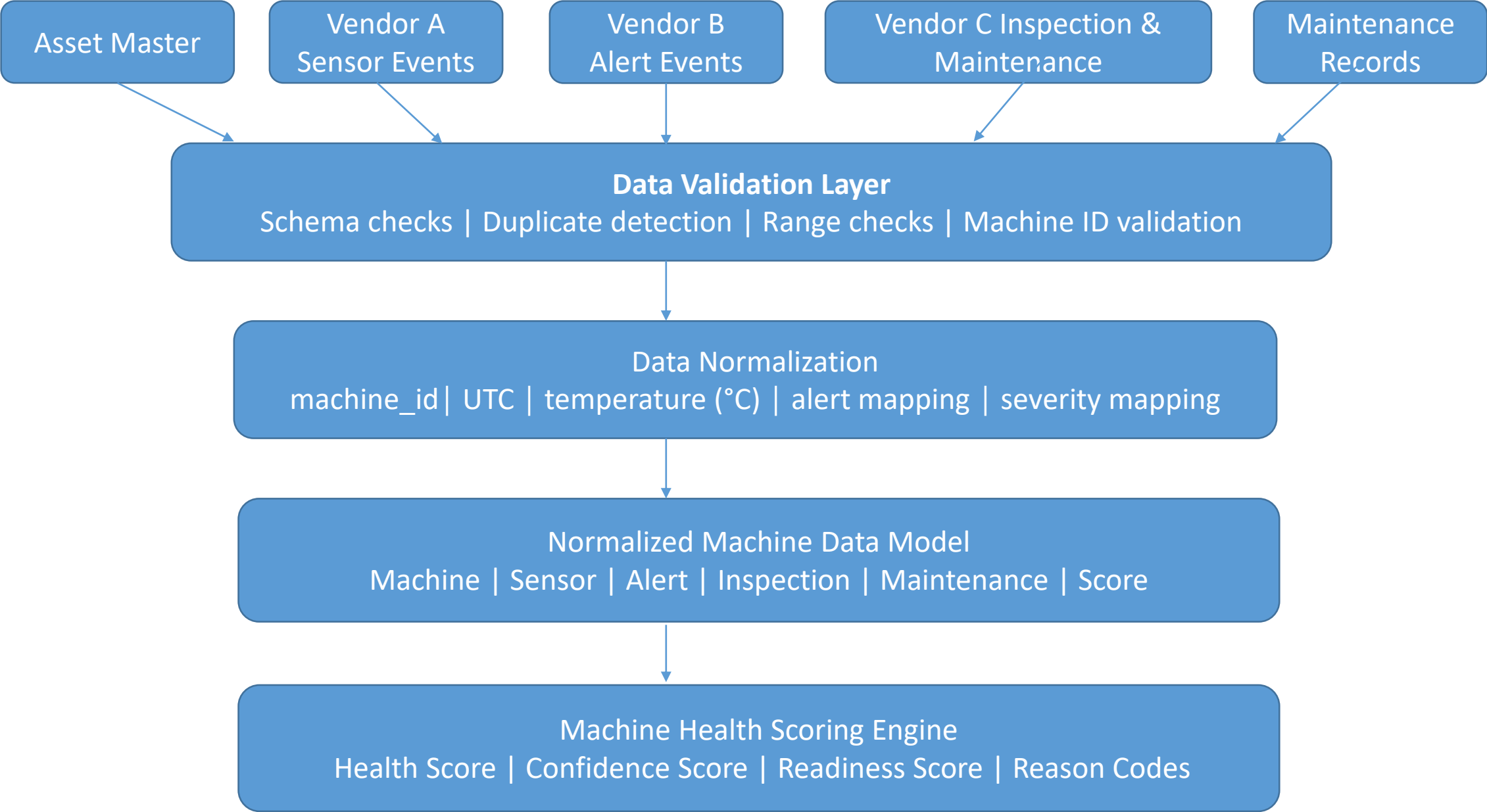
- Data Integration – Integrate data from multiple vendor APIs.
- Data Normalization – Standardize schemas, timestamps, and measurement units.
- Machine Health Score – Generate explainable machine health indicators.
- Confidence & Readiness Scores – Measure prediction reliability and data sufficiency.
- Backend APIs – Expose machine and plant insights for downstream systems.
- Future ML – Enable predictive maintenance and Remaining Useful Life (RUL) prediction.

Vendor Data Assessment

Assessment of data quality, usability, transformation requirements, and integration challenges across all vendor datasets. Assessment of data quality, transformation requirements, join strategy, and integration challenges across all vendor datasets.

Dataset	Primary Data	Directly Usable	Transformation Required	Join Key	Challenges
Asset Master	Machine Metadata	Machine Type, Criticality	Validation	machine_id	Missing master values
Vendor A	Sensor Events	Temperature, Vibration	Timestamp validation	machine_id	Missing sensor readings
Vendor B	Alert Events	Alerts, Power	°F→°C, Epoch→UTC, assetCode→machine_id	assetCode → machine_id	Different schema
Vendor C	Inspection	Maintenance, Notes	Normalize Inspection Notes	machine_ref	Unstructured notes
Maintenance	Work Orders	Service History	Date validation	machine_id	Missing records

Normalized Data Model



Machine Health Score Design

Health Score Factors

Sensor Health (40%)

- Temperature
- Vibration

Maintenance (25%)

- Service History

Inspection (20%)

- Inspection Result

Alerts (15%)

- Alert Severity

Health Score Interpretation

Score	Status
80–100	Healthy
60–79	Warning
40–59	High Risk
0–39	Critical

Health Score = 40% Sensor Health + 25% Maintenance + 20% Inspection + 15% Alerts
Weights are business-defined and can be recalibrated using historical machine failure data.

Confidence / Reliability Score

Confidence Score Factors

- Data Quality (40%)
 - Missing Values
 - Invalid Records
- Sensor Reliability (30%)
 - Sensor Health
 - Signal Quality
- Vendor Consistency (20%)
 - Cross-vendor Validation
- Data Freshness (10%)
 - Recent Timestamp

Confidence Score Interpretation

Confidence	Interpretation
90–100	Very High
70–89	High
50–69	Medium
Below 50	Low

Confidence Score = 40% Data Quality + 30% Sensor Reliability + 20% Vendor Consistency + 10% Data Freshness
A low confidence score indicates low trust in the available data, not necessarily poor machine health.

Maturity / Readiness Score

Readiness Score Factors

- Historical Data (40%)
 - Data Availability
- Maintenance History (25%)
 - Service Records
- Inspection Coverage (20%)
 - Inspection Frequency
- Sensor Coverage (15%)
 - Active Sensors

Readiness Score Interpretation

Readiness	Interpretation
80–100	Production Ready
60–79	Sufficient Data
40–59	Limited Data
Below 40	Insufficient Data

Readiness Score = 40% Historical Data + 25% Maintenance History + 20% Inspection Coverage + 15% Sensor Coverage

Confidence Score → Measures Data Reliability

Readiness Score → Measures Data Sufficiency

Feature Engineering & Future ML

Feature Engineering

Derived Features

- Temperature Deviation
- Vibration Trend
- Days Since Last Service
- Alert Frequency
- Inspection Defect Count
- Machine Age
- Operating Hours

Future ML Approach

Target Prediction

- Machine Failure Risk
- Remaining Useful Life (RUL)

Model Candidates

- Random Forest
- XGBoost

Validation

- Time-Based Split
- Cross Validation

Safe Deployment

- Drift Monitoring
- Human Review
- Target: Failure within Next 7 Days

***ML will be introduced only after sufficient historical labeled data becomes available.
Until then, an explainable rule-based scoring approach will be used.***

No Data Leakage
Train using historical data only
Never use future failures

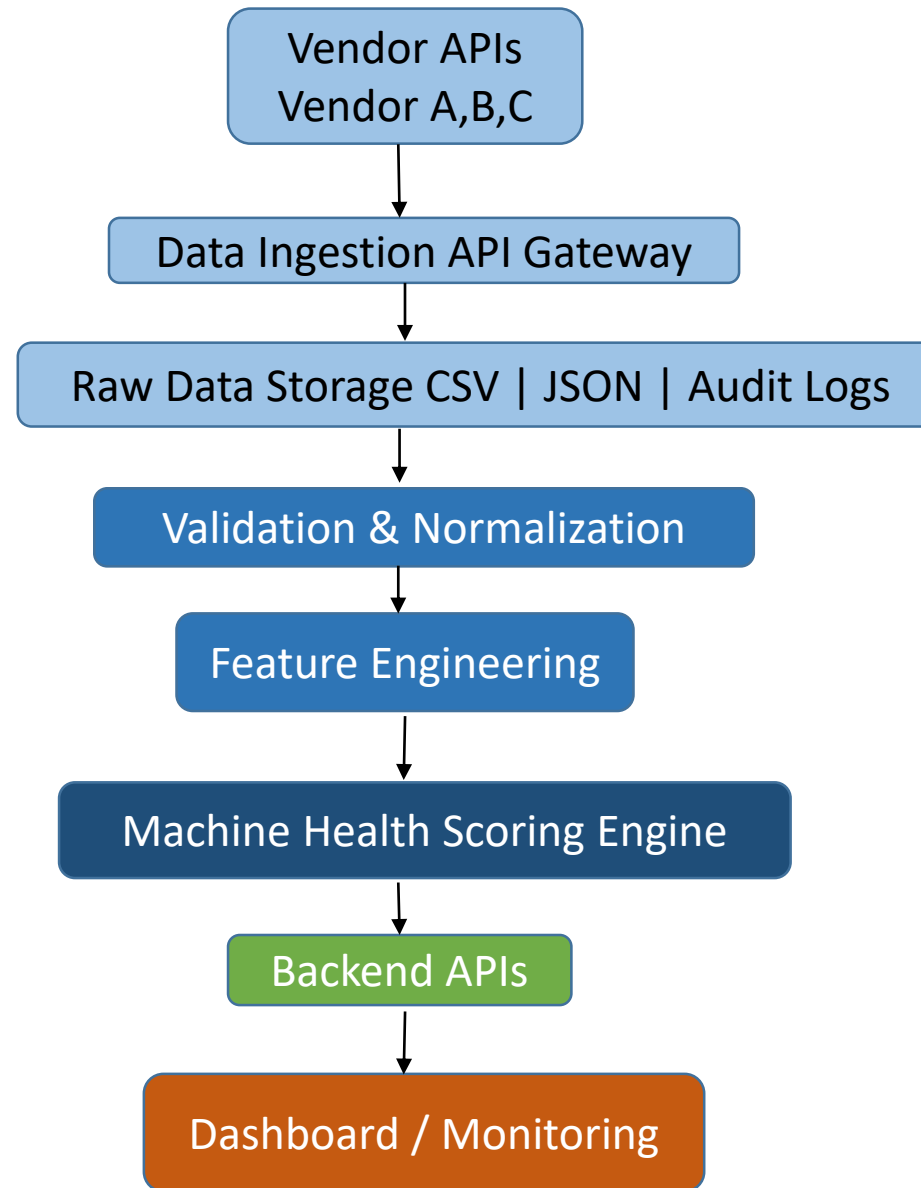
Backend-Facing API Design

Endpoint	Method	Purpose	Output
/api/v1/events/ingest	POST	Ingest vendor data	Trace ID
/api/v1/machines/{machine_id}/score	GET	Get machine score	Health Score, Confidence Score, Readiness Score
/api/v1/lines/{line_id}/summary	GET	Line-level summary	Avg Score, Risk MachinAverage Score, At-Risk Machines
/api/v1/plants/{plant_id}/summary	GET	Plant-level summary	Health Overview

Every API response includes:

- *trace_id*
- *request_id*
- *timestamp*
- *data_version*
- *score_version*
- *error_code*

High-Level System Architecture



***Supports multi-vendor ingestion, traceable processing ,
explainable scoring, API serving, and future ML deployment.***

Conclusion & Future Roadmap

Assessment Outcomes

- ✓ Integrated multi-vendor industrial datasets
- ✓ Designed a normalized machine-level data model
- ✓ Designed an explainable Health, Confidence, and Readiness scoring framework
- ✓ Proposed backend-facing APIs for machine and plant insights
- ✓ Designed a scalable data processing architecture

Future Roadmap

- ✓ Predictive Maintenance using Machine Learning
- ✓ Remaining Useful Life (RUL) Prediction
- ✓ Real-time Streaming (Kafka / MQTT)
- ✓ Automated Anomaly Detection
- ✓ Continuous Model Monitoring, Drift Detection & Retraining

The proposed solution delivers a scalable, explainable, and production-ready data science framework for industrial equipment monitoring across multiple vendors.

Data Quality Challenges

Challenges Identified

- ✓ Missing sensor readings
- ✓ Different machine identifiers
- ✓ Multiple timestamp formats
- ✓ Different measurement units
- ✓ Duplicate records
- ✓ Free-text inspection notes

Business Impact

- ✓ Incorrect machine health score
- ✓ Failed dataset joins
- ✓ Inconsistent reporting
- ✓ Reduced prediction reliability
- ✓ Difficult cross-vendor comparison
- ✓ Increased manual effort

Data quality validation is mandatory before scoring or machine learning.

Data Normalization Rules

Data Issue	Normalization Rule
Temperature (°F)	Convert to Celsius (°C)
Epoch Timestamp	Convert to UTC Timestamp
assetCode → machine_id	Map to machine_id
Alert Severity (Text)	Convert to Numeric Severity (1–5)
Different Vendor Schemas	Map to Common Standard Schema
Free-text Inspection Notes	Standardize using predefined categories

All vendor datasets are transformed into a common machine-level schema before feature engineering and scoring.

Design Assumptions & Key Decisions

Design Assumptions

- ✓ machine_id is the primary unique identifier
- ✓ Asset Master is the source of truth
- ✓ Missing values are validated before scoring
- ✓ Vendor data may arrive at different frequencies
- ✓ Scores are recalculated after every successful data ingestion

Key Design Decisions

- ✓ Explainable rule-based scoring for transparency
- ✓ Common normalized schema across all vendors
- ✓ Modular API-based architecture
- ✓ ML introduced only after sufficient labeled data
- ✓ Versioned scoring to support future enhancements

The solution prioritizes explainability, scalability, and maintainability while remaining ready for future machine learning integration.

Sample Machine Output (API Response)

```
GET /api/v1/machines/{machine_id}/score
{
  "machine_id": "MCH-001",
  "health_score": 86,
  "confidence_score": 92,
  "readiness_score": 84,
  "status": "Healthy",
  "reason_codes": [
    "Temperature within threshold",
    "Recent maintenance completed",
    "No critical alerts detected"
  ],
  "last_updated": "2026-08-05T13:10:00Z",
  "score_version": "1.0"
}
```

The API provides explainable machine health insights with confidence, readiness, reason codes, and versioning to support reliable backend integration.